Lecture 7 - REST and Hospital CRUD APIs

Friday, October 16, 2026 8:13 PM

Lecture 7 — REST and Hospital CRUD APIs

Design → build → predict → send → inspect → verify stored state

1. REST, API approaches and the Hospital contract

API: the common starting point

An API defines how another program interacts with an application: available operations, accepted inputs and returned results. REST, RPC, SOAP and GraphQL describe different aspects or approaches to that interaction. They are not versions of Spring Boot.

Common example: retrieve the details of patient7, named Asha, aged21. The examples below illustrate alternative contracts for that same requirement.

REST — resources and a uniform interface

REST stands for Representational State Transfer. It is an architectural style. In a resource-oriented HTTP API, a URL identifies a resource and standard HTTP method semantics describe the operation. A representation carries the resource data. JSON is the chosen format in this application; REST does not require JSON.

```
GET /patients/7

{
  "id": 7,
  "name": "Asha",
  "age": 21
}
```

In this example, /patients/7 identifies the patient resource, GET requests retrieval, and the JSON body is a representation of the patient.

```
GET    /patients/7   → retrieve patient7
PUT    /patients/7   → replace permitted patient details
DELETE /patients/7   → remove patient7
POST   /patients     → create a patient in the collection
```

Stateless interaction means each request contains the information needed to understand it instead of depending on stored client conversational context. DELETE /patients/7 identifies its target directly. The application can still store patient records in a map or database; resource data is distinct from client session context.

REST is broader than URL naming. Its formal constraints include client/server separation, statelessness, caching, a uniform interface, layering and optional code-on-demand. The classroom application demonstrates resource-oriented HTTP basics; an annotation or JSON response alone does not guarantee REST conformance.

RPC — invoking a remote operation

RPC stands for Remote Procedure Call. Its contract emphasizes executing a named operation with arguments in another process or application. A client-side API can resemble a local method call while the framework transports the request over a network.

```
// Familiar local call:
patientService.getPatient(7);

// Illustrative RPC-style message, not a universal RPC wire format:
{
  "method": "getPatient",
  "params": {
    "id": 7
  }
}
```

The operation name is getPatient and the argument is7. A resource-oriented HTTP description instead uses GET /patients/7. The emphasis differs: operation invocation versus resource interaction through standard semantics. URL wording alone does not determine the architecture, and some domain actions do not fit simple CRUD operations.

gRPC — a specific RPC framework

gRPC is a framework for implementing RPC services. Service operations and messages are commonly defined using Protocol Buffers; tooling generates client and server code. Native gRPC uses HTTP/2 transport. RPC is the approach; gRPC is one implementation framework.

```
// Illustrative generated-client call:
patientClient.getPatient(request);
```

The client call represents a request to a remote service. A JSON HTTP endpoint named /getPatient is not automatically a gRPC service.

SOAP — structured XML messages

SOAP is a protocol for exchanging structured messages using XML. A SOAP message has an Envelope containing a Body and can have an optional Header. It can operate over different underlying protocols, including HTTP. SOAP is not simply a REST response written in XML.

```
<!-- XML data example -->
<patient>
    <id>7</id>
    <name>Asha</name>
</patient>
```

```
<!-- Illustrative SOAP request message -->
<soap:Envelope
    xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
    xmlns:p="https://example.com/patients">
    <soap:Body>
        <p:GetPatient>
            <p:id>7</p:id>
        </p:GetPatient>
    </soap:Body>
</soap:Envelope>
```

Envelope provides the outer SOAP message structure. Body carries the application message. GetPatient and id belong to this illustrative service contract. XML alone is not SOAP; the SOAP structure and rules distinguish the message. SOAP can support RPC-style interactions, so SOAP and RPC are not mutually exclusive categories.

GraphQL — selecting fields from an API schema

GraphQL is a query language for APIs and a server-side execution runtime. A schema defines the types, fields and operations available to clients. A query selects fields supported by that schema; backend resolver code obtains the data and enforces application rules.

Assuming an illustrative schema exposes patient(id), a name-only query can be written as follows.

```
query {
  patient(id: 7) {
    name
  }
}
```

```
{
  "data": {
    "patient": {
      "name": "Asha"
    }
  }
}
```

Adding age to the selection requests that field as well.

```
query {
  patient(id: 7) {
```

```
    name
    age
  }
}
```

GraphQL does not send raw SQL directly to the database or permit access to arbitrary hidden fields. Its server still implements retrieval and authorization. REST APIs can also offer field-selection parameters; GraphQL makes selection part of its query language.

Comparison using the same patient lookup

SAME NEED: GET DETAILS FOR PATIENT 7

Approach	Main idea	Illustrative request
REST	Address a resource; use HTTP semantics	GET /patients/7
RPC / gRPC	Invoke a named operation; gRPC is an RPC framework	getPatient(id = 7)
SOAP	Structured XML Envelope carrying a message	GetPatient request inside a SOAP Envelope
GraphQL	Ask a schema for selected response fields	{ patient(id: 7) { name } }

```
REST      Architectural style       Resource: GET /patients/7
RPC       Remote-call approach      Operation: getPatient(id=7)
gRPC      RPC framework             Typed generated service call
SOAP      XML messaging protocol    Message inside a SOAP Envelope
GraphQL   Query language/runtime    Selected fields from an API schema
```

These are not five equivalent categories or versions of one technology. Their use depends on the required interaction and ecosystem. HTTP and JSON may appear in several approaches. The Hospital implementation uses resource-oriented HTTP endpoints with Spring MVC.

Hospital API contract

```
GET /patients            → 200, patient array
GET /patients/{id}       → 200 patient or404
POST /patients           → name/age JSON;201 plus generated ID
PUT /patients/{id}       → full name/age replacement;200 or404
DELETE /patients/{id}    → 204 without body or404
GET /patients/search?name=... → 200, matching array
```

Patient identifiers appear in resource paths. Create/update details are carried in request bodies. A search name is a query parameter. These are contract examples; Java controller implementation follows in later sections.

References

REST: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm | RPC/gRPC: https://grpc.io/docs/what-is-grpc/introduction/ | SOAP: https://www.w3.org/TR/soap12/ | GraphQL: https://graphql.org/learn/introduction/

2. Base mapping

```
@RestController
@RequestMapping("/patients")
public class PatientController {
    // Add mapped methods here.
}
```

What is the complete URL path for @GetMapping("/{id}") inside this class?

If the Java method name changes, must its URL change?

Which method starts the application? Which methods handle later requests?

3. Patient model and JSON responses

```
package com.accio.lecture7;
public class Patient {
    private final long id;
    private final String name;
    private final int age;
    public Patient(long id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }
    public long getId() { return id; }
    public String getName() { return name; }
    public int getAge() { return age; }
}
```

```
@GetMapping("/preview")
public Patient preview() {
    return new Patient(7, "Asha", 21);
}
```

TRACE THE RESPONSE

Java: new Patient(7, "Asha", 21)

HTTP body: _____

Content-Type: _____

Browser receives: Java reference, JSON data, or an HTML page?

Which code constructs the object? Where does that code execute?

Do we need to concatenate JSON or call toString()?

What would change if the method returned a List<Patient>? An empty list?

Does returning this object save a patient anywhere?

4. Choose status, headers and body with ResponseEntity

Step 1. Import ResponseEntity and replace the existing preview method

```
// Add this import above the existing controller:
import org.springframework.http.ResponseEntity;

// REPLACE the preview method from point 3; do not duplicate its mappi
ng.
@GetMapping("/preview")
public ResponseEntity<Patient> preview() {
    Patient patient = new Patient(7, "Asha", 21);
    return ResponseEntity.ok(patient);
}
```

Step 2. Run the preview and inspect the response

```
GET  http://127.0.0.1:8080/patients/preview

HTTP/1.1 200 OK
Content-Type: application/json

{"id":7,"name":"Asha","age":21}
```

Compare the response with point3. Did the JSON become a nested ResponseEntity object? Which parts of HTTP are outside the body?

Step 3. Read the response forms we will use next

```
// Syntax reference, NOT extra statements to paste into the class body
:
ResponseEntity.ok(patient);                     // 200 + body
ResponseEntity.status(404).body(error);          // chosen status + er
ror body
ResponseEntity.created(location).body(patient); // 201 + Location + bo
dy
ResponseEntity.badRequest().body(error);        // 400 + error body
ResponseEntity.noContent().build();             // 204, no body
```

Is ResponseEntity the same thing as the Patient domain model? Does choosing201 save a record?

Which response form would you choose for a missing record, creation, invalid input and successful deletion?

5. In-memory storage: build and test in stages

Keep Patient.java and the existing application main. Use one PatientController. Match the package to your current project.

FOLLOW THE SAME RUNNING APPLICATION

patients map: 1 → Asha, 21 | 2 → Rahul, 30

GET /patients → body: _____

GET /patients/1 → body: _____

GET /patients/999 → status: _____ body: _____

Restart the application → what remains?

A. PatientController.java — seed the map and add GET-all

```
package com.accio.lecture7;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/patients")
public class PatientController {
    private final Map<Long, Patient> patients = new LinkedHashMap<>();
    private long nextId = 3; // Used later when POST creates a patient
.

    public PatientController() {
        patients.put(1L, new Patient(1, "Asha", 21));
        patients.put(2L, new Patient(2, "Rahul", 30));
    }

    @GetMapping
    public ResponseEntity<List<Patient>> getAll() {
        return ResponseEntity.ok(new ArrayList<>(patients.values()));
    }
}
```

GET http://127.0.0.1:8080/patients

What will the response body look like? What does final prevent here? What changes if the map is built inside getAll()?

B. ApiError.java — create the response model before using it

```
package com.accio.lecture7;
public class ApiError {
    private final String message;
    public ApiError(String message) { this.message = message; }
    public String getMessage() { return message; }
}
```

Which JSON property will this object expose?

C. Confirm imports, then add GET-by-ID inside the same controller

```
// Add these imports above the class:
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PathVariable;


@GetMapping("/{id}")
public ResponseEntity<Object> getOne(@PathVariable("id") long id) {
    Patient patient = patients.get(id);
    if (patient == null) {
        return ResponseEntity.status(404).body(new ApiError("Patient n
ot found"));
    }
    return ResponseEntity.ok(patient);
}
```

Where does the lookup happen? Which line chooses the missing-record response?

D. Test each route and inspect the response

GET http://127.0.0.1:8080/patients
GET http://127.0.0.1:8080/patients/1
GET http://127.0.0.1:8080/patients/999
GET http://127.0.0.1:8080/patients/abc

Predict status, object versus array, and missing-data behavior before sending each request.

Which requests read stored objects? Which failure happens before our method runs? What remains after a restart?

6. POST: write and run in this order

Step 1. Create PatientInput.java as a separate class

```
package com.accio.lecture7;
public class PatientInput {
    private String name;
    private Integer age;
    public PatientInput() { }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public Integer getAge() { return age; }
    public void setAge(Integer age) { this.age = age; }
}
```

Step 2. Add imports to the existing PatientController

```
import java.net.URI;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
```

Keep the existing ResponseEntity import, map, GET methods and nextId = 3 field from point 5. Do not redeclare nextId or create a second controller.

Step 3. Add one POST method inside PatientController

```
@PostMapping
public ResponseEntity<Object> create(@RequestBody PatientInput input)
{
    long id = nextId++;
    Patient patient = new Patient(id, input.getName(), input.getAge())
;
    patients.put(id, patient);
    return ResponseEntity.created(URI.create("/patients/" + id)).body(
patient);
}
```

Step 4. Run the updated application, then send JSON

```
POST  http://127.0.0.1:8080/patients
Content-Type: application/json

{"name":"Meera","age":26}
```

Postman: POST → /patients → Body → raw → JSON. Verify Content-Type: application/json. Run/restart the updated application before sending.

Step 5. Read back the created patient

```
Record response status: ______
Record Location: _____________
Record generated ID: _________

GET /patients/{returnedId}
GET /patients
```

Which line changes stored data? Which parts are Spring conversion and which are our code?

Predict what happens to the ID and collection if the same POST is sent again.

7. PUT: add and test in order

PREDICT ID, VALUES AND COLLECTION SIZE

Before: ID 3 | Meera | age 26

PUT /patients/3 body: {"name":"Meera Shah","age":27}

After first request: ID ____ | name _____ | age ____ | count ____

After repeating the same PUT: _____

Missing ID? Missing field? Partial update via PATCH?

Step 1. Add PutMapping import above PatientController

```
import org.springframework.web.bind.annotation.PutMapping;
```

Reuse existing PathVariable, RequestBody and ResponseEntity imports, PatientInput, Patient, ApiError and the patients map. Do not create another input class or nextId counter.

Step 2. Add one replace method inside the existing controller

```
@PutMapping("/{id}")
public ResponseEntity<Object> replace(@PathVariable("id") long id,
                                      @RequestBody PatientInput input)
{
    // 1. Reject an ID that is not stored.
    if (!patients.containsKey(id)) {
        return ResponseEntity.status(404).body(new ApiError("Patient n
ot found"));
    }
    // 2. Keep the path ID and replace the permitted details.
    Patient replacement = new Patient(id, input.getName(), input.getAg
e());
    patients.put(id, replacement);
    // 3. Return the updated stored representation.
    return ResponseEntity.ok(replacement);
}
```

Step 3. Prepare and send a full PUT request

```
PUT  http://127.0.0.1:8080/patients/3
Content-Type: application/json

{"name":"Meera Shah","age":27}
```

Use an existing ID returned by point 6 if it differs from 3. Postman: PUT → correct patient URL → Body → raw → JSON → Send. Restart the edited application before testing; if restart resets the map, create the patient again first.

Step 4. Retrieve and repeat — verify values and identity

```
GET /patients/{updatedId}
GET /patients
Repeat the SAME PUT with the SAME body
GET /patients/{updatedId}
GET /patients
```

Predict the ID, stored values and collection size after each request. Does repeated PUT add

another patient?

Step 5. Try a missing ID with a valid full body

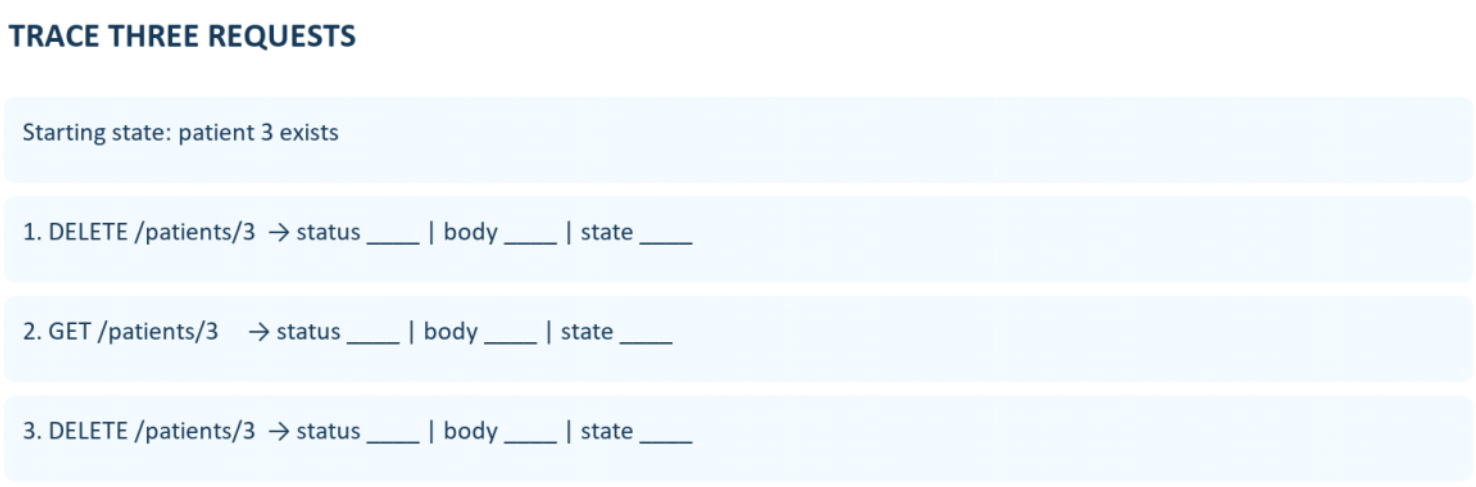
```
PUT /patients/999
Content-Type: application/json

{"name":"Nobody","age":40}

GET /patients/999
GET /patients
```

What is the status? Was patient999 created? Which branch determines this contract?

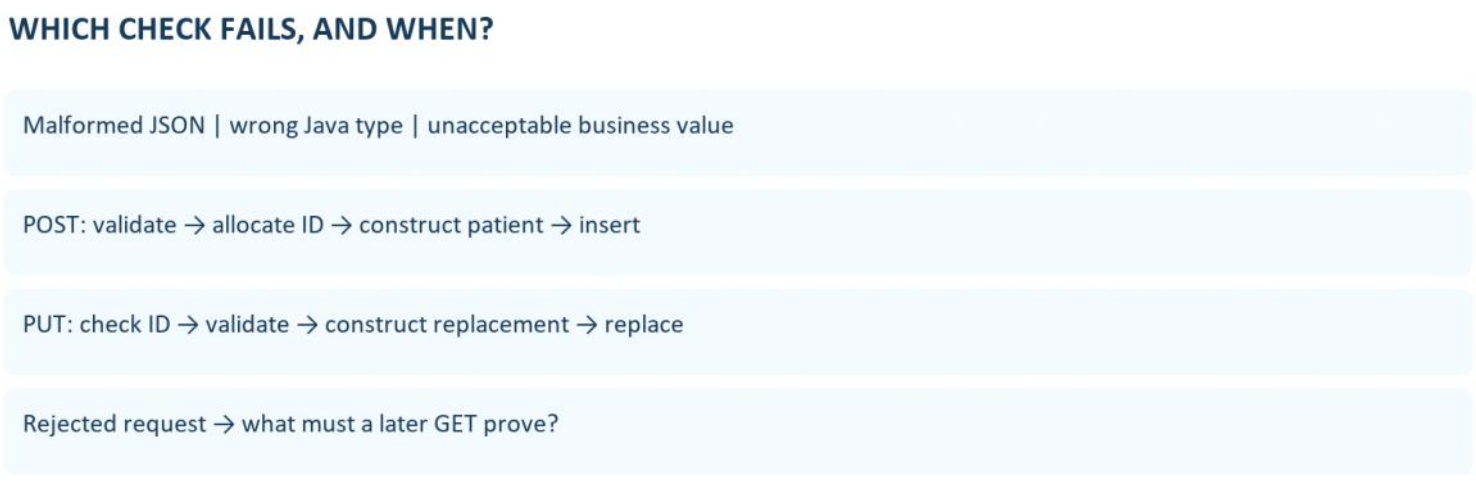
8. DELETE and repeated requests



Can different response statuses still leave the same final resource state?

Should the next POST necessarily reuse the deleted ID?

9. Validate before changing state




```
Use POST /patients, one case at a time:
{"name":"","age":21}
{"name":"   ","age":21}
{"age":21}
{"name":"Asha"}
{"name":"Asha","age":-1}
```

```
{"name":"Asha","age":121}
{"name":" Baby ","age":0}

For each: predict status, message and whether stored state changes.
```

Class rule: nonblank name; age required and between 0 and 120. Trim accepted names.

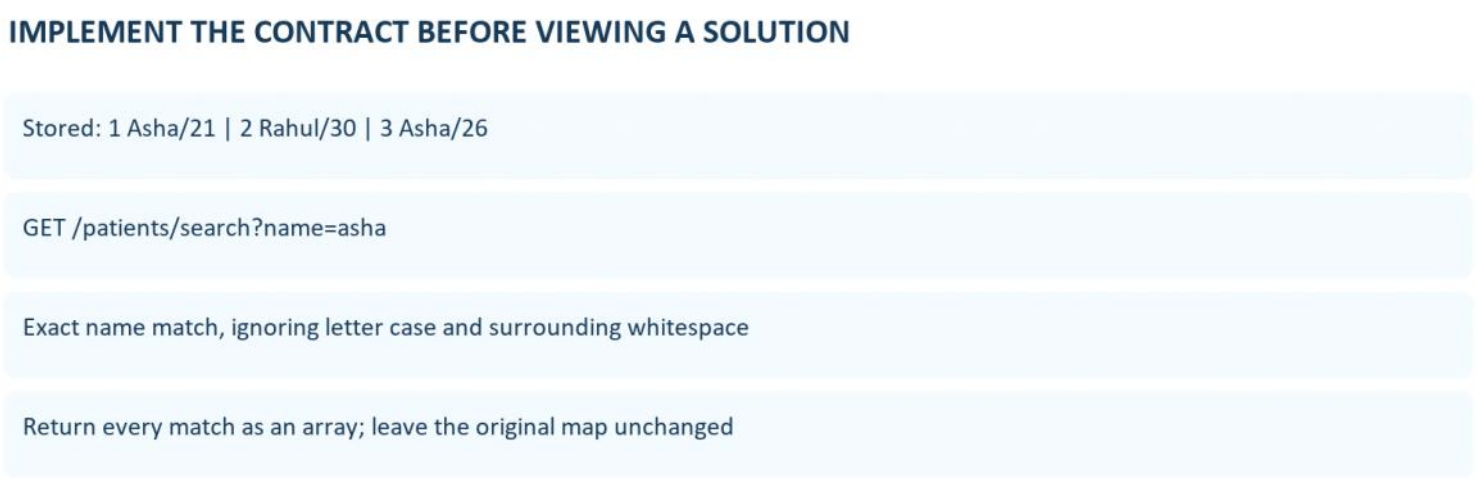
Should a rejected POST consume the next ID?

Can valid JSON contain an invalid age?

Attempt a PUT with a changed name and invalid age. Then GET the patient. What stayed unchanged?

Do malformed JSON and our business-rule failures necessarily have identical error bodies?

10. Independent exercise: search patients



Write the endpoint using the current map, @RequestParam and a loop.

Test two matching names, no match, missing name and the unchanged full collection.

Why could returning from the first loop iteration be wrong?

Which outcome would you choose when the endpoint exists but nothing matches?

11. Independent lifecycle test

```
Restart to a known state; record the generated ID.
1. Read the collection.
2. Create a patient.
3. Retrieve the returned ID.
4. Update the full details.
5. Retrieve and compare ID and values.
6. Repeat the same PUT.
7. Try an invalid update, then retrieve again.
8. Delete the patient.
9. Retrieve and delete that ID again.
10. Repeat a valid POST twice; compare generated IDs.
```

For each request record method, URL, status, important body values and stored-state effect.

Which failures came from our controller? Which happened before it ran?

12. Sorting and pagination: add, replace and test in stages

SORT FIRST, THEN RETURN A SLICE

Stored: ID1 age30 | ID2 age20 | ID3 age20 | ID4 age40

Sorted by age, then ID: _____

offset=0, limit=2 → returned IDs _____

offset=2, limit=2 → returned IDs _____

offset=999, limit=2 → JSON response: _____

Step 1. PatientController.java — add/confirm imports

```
import java.util.Comparator;
import org.springframework.web.bind.annotation.RequestParam;
// Already used earlier: ArrayList, List, ResponseEntity, GetMapping.
// Reuse existing Patient, ApiError and the patients map.
```

Keep all previous endpoints. Add one browse method inside the existing controller.

Step 2. First add an age-sorted GET /patients/browse

Try sorting all current patients by age with the existing Comparator knowledge before looking at the code below.

```
@GetMapping("/browse")
public ResponseEntity<List<Patient>> browse() {
    List<Patient> ordered = new ArrayList<>(patients.values());
    ordered.sort(Comparator.comparingInt((Patient p) -> p.getAge())
            .thenComparingLong((Patient p) -> p.getId()));
    return ResponseEntity.ok(ordered);
}
```

```
GET /patients/browse
GET /patients
```

Step 3. Replace browse with a sortBy choice — still no pagination

```
@GetMapping("/browse")
public ResponseEntity<Object> browse(
        @RequestParam(name = "sortBy", defaultValue = "id") String sor
tBy) {
    List<Patient> ordered = new ArrayList<>(patients.values());
    if (sortBy.equals("age")) {
        ordered.sort(Comparator.comparingInt((Patient p) -> p.getAge()
)
                .thenComparingLong((Patient p) -> p.getId()));
    } else if (sortBy.equals("id")) {
        ordered.sort(Comparator.comparingLong((Patient p) -> p.getId()
));
    } else {
        return ResponseEntity.badRequest().body(new ApiError("sortBy m
ust be id or age"));
    }
    return ResponseEntity.ok(ordered);
}
```

```
GET /patients/browse?sortBy=age
```

```
GET /patients/browse?sortBy=id
GET /patients/browse
GET /patients/browse?sortBy=unknown
```

Predict order and status before sending. What happens when sortBy is omitted?

Step 4. Trace offset and limit before changing code

```
Already sorted IDs: [2, 3, 1, 4]
offset = 0, limit = 2 → positions [0, 2)
offset = 2, limit = 2 → positions [2, 4)
offset = 999 → start must be clamped to collection size

total = ordered.size()
start = Math.min(offset, total)
end = start + Math.min(limit, total - start)
```

Which IDs are on each page? Why sort before slicing? What should an offset beyond the end return?

Step 5. Replace only browse with the complete paginated version

```
@GetMapping("/browse")
public ResponseEntity<Object> browse(
        @RequestParam(name = "sortBy", defaultValue = "id") String sor
tBy,
        @RequestParam(name = "offset", defaultValue = "0") int offset,
        @RequestParam(name = "limit", defaultValue = "5") int limit) {
    // 1. Reject invalid page inputs before using them.
    if (offset < 0 || limit < 1 || limit > 50) {
        return ResponseEntity.badRequest().body(new ApiError("Invalid
offset or limit"));
    }
    // 2. Copy the values, then sort before taking a slice.
    List<Patient> ordered = new ArrayList<>(patients.values());
    if (sortBy.equals("age")) {
        ordered.sort(Comparator.comparingInt((Patient p) -> p.getAge()
)
                .thenComparingLong((Patient p) -> p.getId()));
    } else if (sortBy.equals("id")) {
        ordered.sort(Comparator.comparingLong((Patient p) -> p.getId()
));
    } else {
        return ResponseEntity.badRequest().body(new ApiError("sortBy m
ust be id or age"));
    }
    // 3. Compute safe bounds; subList end is exclusive.
    int total = ordered.size();
    int start = Math.min(offset, total);
    int end = start + Math.min(limit, total - start);
    List<Patient> items = new ArrayList<>
(ordered.subList(start, end));
    // 4. Return only the selected patients as a JSON array.
    return ResponseEntity.ok(items);
}
```

Step 6. Run consecutive pages and error cases

```
GET /patients/browse?sortBy=age&offset=0&limit=2
GET /patients/browse?sortBy=age&offset=2&limit=2
GET /patients/browse?offset=999&limit=2
GET /patients/browse?offset=-1
GET /patients/browse?limit=0
GET /patients/browse?limit=51
GET /patients/browse?sortBy=unknown
GET /patients/browse?offset=abc
GET /patients
```

Create at least five patients, including equal ages. Record current IDs first. Verify consecutive JSON-array slices, an offset beyond the end, invalid inputs and the unchanged full collection.

Optional extension — only if there is time

Add a descending boolean query choice and reverse the selected Comparator before slicing. Keep default ascending. Predict how ties reverse as well. This is outside the main timing allocation; do not introduce Pageable, streams, JPA or a UI.

13. Recap

Trace one create request from HTTP input to stored data and response.

What changed between a preview endpoint and this CRUD application?

Why does the data disappear after restarting?

Which repeated operation can create duplicates under our contract?

What responsibilities have accumulated in PatientController?

Optional practice: age-range filter

Implement GET /patients/by-age?min=18&max=30 using an inclusive age range.

Reject invalid bounds with 400; return all matching patients, including an empty result list.

Optional independent in-class challenge — Delivery quotes, 30-minute target

A delivery service offers standard delivery at 40 units/kg and express at 70 units/kg. Implement the following two APIs. You choose sensible Java class names, JSON field names and model fields; document your choices before coding. Use separate request and response models rather than returning the input unchanged.

```
GET /delivery/options
Return both delivery modes and their per-
kilogram rates as a JSON array.

POST /delivery/quote
Accept a delivery mode and parcel weight in kilograms as JSON.
Return JSON with the mode, weight and calculated delivery charge.
Charge = rate per kilogram × weight.
This calculates a quote; it does not save an order.
```

Requirements: successful responses use200. Reject missing/blank/unknown mode and missing/nonpositive weight with400 and a message. Accept mode names without case sensitivity. No ID, database, collection of orders, layers, service beans or UI is required. Keep the Hospital endpoints working.

Plan the models before implementing the APIs

```
Request model fields: _____________________________
Options-response fields: ___________________________
Quote-response fields: _____________________________
Example POST JSON: _________________________________
Example successful response: _______________________
```

Your response must expose the computed charge; the client should not supply the final charge. A dedicated simple error model may be used, or reuse the message model already available in your project.

Acceptance checks

```
GET options: both modes and their rates
Standard, 2.5kg → calculate and verify charge
Express, 2kg → calculate and verify charge
Change only the weight → verify that the response changes
Unknown mode → reject with a message
Zero/negative/missing weight → reject without a server error
Existing Hospital GET endpoint still works
```

Target: 4 minutes model design, 18 implementation, 5 request checks, 3 review. Use the existing project; no new project setup.

Homework — Independent Watchlist CRUD API

Movie fields: id, title, releaseYear. Server assigns the ID. Store movies in memory.

```
GET /movies
GET /movies/{id}
POST /movies
PUT /movies/{id}
DELETE /movies/{id}

Create/update JSON:
{"title":"Interstellar","releaseYear":2014}
```

Rules: required nonblank title; releaseYear required, 1888–2100 inclusive. PUT replaces existing details and retains the ID. Invalid writes must leave existing data unchanged. Duplicate titles are allowed.

Use 201 plus Location for creation; 204 without a body for successful deletion; 404 for missing IDs; 400 for invalid input. Do not allocate IDs with collection size + 1.

Submit source and request evidence: create/read/update/delete; rejected update followed by unchanged GET; delete an early ID then create without overwriting another movie; empty collection; repeated POST versus PUT.

Bring the application ready to run for Lecture 9. Optional search/sorting is extra. No database, UI, layers or dependency injection requirement for this homework.